

LAPORAN PRATIKUM

STRUKTUR DATA

“ADT Jam pada Bahasa Pemrograman Java”

disusun Oleh:

Muhammad Yasin Habiburrahman

2511532016

Dosen Pengampu:

Dr. Wahyudi. S.T.M.T

Asisten Pratikum:

SHERLY SUKMADIRA PUTRI



DEPARTEMEN INFORMATIKA FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman.

Tugas kelompok ini kami susun dengan tujuan untuk memenuhi salah satu tugas mata kuliah Algoritma dan Pemrograman, dengan tema “ADT Jam pada Bahasa Pemrograman Java”, yang mencakup bagaimana mengonstruksi sistem dari sebuah jam melalui bahasa pemrograman Java.

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pembimbing yang telah memberikan arahan, serta kepada seluruh anggota kelompok yang telah bekerja sama dengan baik sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami khususnya, dan bagi pembaca pada umumnya.

Padang, 1 April 2026

Muhammad Yasin Habiburrahman

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN.....	2
2.1 <i>Abstract Data Type</i> (ADT)	2
2.2 Struktur ADT Jam pada Program Java	2
2.2.1 Atribut	3
2.2.2 Konstruktor	3
2.2.3 Validator	3
2.2.4 Selektor	3
2.2.5 Mutator.....	4
2.3 Fungsi-Fungsi Penunjang ADT Jam.....	4
2.3.1 Konversi	4
2.3.2 Relasional.....	5
2.3.3 Fungsi Aritmatika	5
2.3.4 Fungsi Durasi	6
2.3.5 Fungsi ToString	6
2.4 Kelas Driver Jam.....	7
2.5 Kelas Driver Lanjutan.....	8
2.5.1 Fungsi Pembuat Jam dari Input.....	8
2.5.2 Blok Program Inisialisasi Dua Objek Jam	8
2.5.3 Blok Program Menampilkan Data	9
2.5.4 Blok Program Operasi Rasional (Perbandingan)	9
2.5.5 Blok Operasi Aritmatika	9
2.6 Latihan Membuat ADT Mobil	10
2.6.1 Atribut	10
2.6.2 Validator dan Konstruktor	10
2.6.3 Selektor dan Mutator.....	11

2.6.4 Method-Method Tambahan.....	11
2.7 Kelas Driver MobilDriver_2511532016.....	12
BAB III KESIMPULAN.....	13
DAFTAR PUSTAKA.....	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengelolaan data merupakan hal yang sangat penting agar suatu sistem dapat berjalan secara terstruktur dan mudah dipelihara. Salah satu konsep yang digunakan untuk mengorganisasi data tersebut adalah Abstract Data Type (ADT). ADT memungkinkan programmer mendefinisikan suatu tipe data beserta operasi-operasi yang dapat dilakukan terhadap data tersebut tanpa harus memperhatikan secara langsung implementasi internalnya.

Salah satu contoh penerapan ADT yang sederhana adalah tipe data Jam yang merepresentasikan waktu dalam satu hari. Data waktu biasanya terdiri dari tiga komponen utama, yaitu jam (hour), menit (minute), dan detik (second). Representasi waktu seperti ini sering digunakan dalam berbagai aplikasi, seperti sistem penjadwalan, pencatatan waktu proses program, sistem absensi, maupun sistem pengelolaan aktivitas lainnya.

Pada praktikum ini dilakukan perancangan dan implementasi ADT Jam menggunakan bahasa pemrograman Java. Melalui implementasi ini, mahasiswa dapat memahami bagaimana konsep ADT diterapkan dalam bentuk kelas (class) dalam pemrograman berorientasi objek.

1.2 Tujuan

1. Memahami konsep Abstract Data Type (ADT) dalam pemrograman.
2. Mempelajari cara merepresentasikan suatu tipe data dalam bentuk class pada bahasa Java.
3. Mengimplementasikan operasi-operasi dasar pada tipe data Jam, seperti selektor, mutator, serta operasi aritmatika terhadap waktu.
4. Melatih kemampuan dalam merancang program yang terstruktur dan modular.

1.3 Manfaat Praktikum

1. Mahasiswa dapat memahami konsep **abstraksi data** dalam pemrograman.
2. Mahasiswa dapat mengimplementasikan ADT menggunakan **bahasa pemrograman Java**.
3. Mahasiswa dapat memahami cara melakukan manipulasi data waktu seperti perbandingan waktu, penjumlahan waktu, serta konversi waktu ke dalam satuan detik.
4. Mahasiswa dapat meningkatkan kemampuan dalam merancang program yang lebih terstruktur dan mudah dikembangkan.

BAB II

PEMBAHASAN

2.1 *Abstract Data Type* (ADT)

ADT adalah tipe data yang didefinisikan berdasarkan data dan operasi yang dapat dilakukan terhadap data tersebut. Hal yang difokuskan adalah apa yang bisa dilakukan, bukan bagaimana implementasinya.

ADT memiliki empat *method* dasar; yaitu konstruktor, validator, selektor, dan mutator. Konstruktor digunakan untuk membuat objek baru dari kelas ADT tersebut. Validator digunakan untuk memastikan apakah suatu jam memiliki waktu yang valid atau tidak. Selektor digunakan untuk mengambil nilai pada objek kelas ADT. Mutator digunakan untuk mengubah atribut dari objek tersebut.

Dalam ADT, ada dua hal yang dibedakan, yaitu spesifikasi, dan implementasi. Spesifikasi terdiri dari data apa saja yang akan direpresentasikan, operasi apa yang tersedia, dan hasil apa saja yang diinginkan dari operasi tersebut. Spesifikasi tidak menjelaskan bagaimana sebuah program bekerja secara internal. Misalkan terdapat sebuah jam, kita tahu bahwa sebuah jam memiliki tiga satuan waktu utama yang terdiri dari jam, menit, dan detik.

Ketika seseorang melihat jam dinding atau jam tangan mereka, informasi yang mereka dapatkan adalah informasi mengenai waktu saat itu. Maka dari itu, mereka melakukan operasi/tindakan untuk mendapatkan informasi mengenai waktu saat itu.

Contoh lain adalah ketika seseorang mengatur ulang waktu pada jam, misalnya setelah mengganti baterai. Tindakan tersebut merupakan operasi yang mengubah nilai waktu yang tersimpan pada jam. Hal inilah yang dikatakan operasi.

2.2 Struktur ADT Jam pada Program Java

Terdapat beberapa *method* esensial agar program dapat berjalan dengan benar. Pada dasarnya, ADT memiliki beberapa operasi-operasi dasar, yaitu konstruktor, validator, selektor, dan mutator.

Berikut ini adalah penjelasan *methode-methode* dasar pada ADT jam.

1. Konstruktor : Membuat objek jam baru
2. Validator : Memastikan suatu jam memiliki waktu yang sah atau tidak
3. Selektor : Mengembalikan nilai jam, menit, atau waktu pada jam
4. Mutator : Mengubah nilai jam, menit, atau waktu pada jam

2.2.1 Atribut

Atribut utama dalam sebuah jam adalah jam, menit dan waktu yang ditampilkan oleh sebuah jam.

```
public final class Jam_2511532016 {
    private int hh; //0...23
    private int mm; //0...59
    private int ss; //0...59
```

Kode Program 2.1 – Atribut Utama ADT Jam

2.2.2 Konstruktor

Pada ADT Jam, konstruktor digunakan untuk menginisialisasi nilai jam, menit, dan detik saat objek pertama kali dibuat. Konstruktor menerima tiga parameter, yaitu h, m, dan s, yang masing-masing merepresentasikan nilai jam, menit, dan detik.

```
//---Konstruktor---
public Jam_2511532016(int h, int m, int s){
    this.hh = h; this.mm=m; this.ss=s;
}
```

Kode Program 2.2 – Potongan Program untuk Konstruktor

2.2.3 Validator

Validator dipakai untuk memastikan apakah sebuah objek dari kelas memiliki atribut-atribut yang sah atau tidak. Untuk ADT jam, nilai-nilai yang harus diperiksa adalah nilai dari masing-masing atribut bernilai non-negatif, berada pada rentang 0 sampai 24 untuk jam, dan 0 sampai 60 untuk menit dan detik.

```
//---Validator---
public static boolean isValid(int h, int m, int s){
    return (h>=0 && h<=23) && (m>=0 && m<=59) && (s>=0 && s<=59);
}
```

Kode Program 2.3 – Potongan Program untuk Validator

2.2.4 Selektor

Selektor berguna saat kita ingin menyimpan/mengetahui suatu nilai yang ada pada atribut objek. Pada ADT jam, nilai yang dikembalikan adalah jam, menit, dan detik dari objek jam tersebut.

```
//---Selektor---
public int getHour(){return hh;}
public int getMinute(){return mm;}
public int getSecond(){return ss;}
```

Kode Program 2.4 – Potongan Program untuk Selektor

2.2.5 Mutator

Mutator digunakan untuk mengubah nilai atribut dari suatu objek. Implementasi pada ADT jam digunakan untuk mengubah nilai atribut dari objek jam. Nilai dari jam, menit, dan detik dapat diubah.

Mutator pada ADT jam memiliki tiga buah *method* untuk mengubah nilai dari masing masing atribut utama. Parameter dari masing-masing prosedur menerima nilai baru yang akan menggantikan nilai atribut sebelumnya yang bersesuaian.

```
//---Mutator---
public void setHour(int h){this.hh = h;}
public void setMinute(int m){this.mm = m;}
public void setSecond(int s){this.ss = s;}
```

Kode Program 2.5 – Potongan Program untuk Mutator

2.3 Fungsi-Fungsi Penunjang ADT Jam

Fungsi-fungsi yang dapat ditambahkan untuk sebuah jam antara lain adalah mengonversi menjadi satuan detik, membandingkan waktu satu dan waktu lainnya, menambahkan satuan waktu, dan selisih nilai antara waktu satu dan waktu lainnya.

2.3.1 Konversi

Untuk mengubah jam ke detik, kita cukup mengalikan nilai jam dengan 3600; dan untuk mengubah menit ke detik, kita cukup mengalikan nilai menit dengan 60. Kemudian, setiap hasil konversi dijumlahkan kemudian ditambahkan dengan nilai detik yang tersisa.

```
//Konversi
public int toSeconds(){return hh*3600 + mm*60 + ss;}
```

Kode Program 2.6 – Potongan Kode Program untuk Konversi ke Detik

Selain itu, terdapat method `fromSeconds(int total)` digunakan untuk membuat objek Jam berdasarkan jumlah detik yang diberikan. Parameter `total` menyatakan total detik yang akan dikonversi menjadi satuan jam, menit, dan detik.

```
public static Jam_2511532016 fromSeconds(int total){
    if(total < 0 ) throw new IllegalArgumentException("Detik Negatif!");
    total %= 24*3600;
    int h = total/3600; total %=3600;
    int m = total/60; int s = total%60;
    return new Jam_2511532016(h,m,s);
}
```

Kode Program 2.7 – Potongan Kode Program untuk Membuat Objek Jam dari Satuan Detik

Pada awal method dilakukan pemeriksaan apakah nilai detik bernilai negatif. Jika nilai negatif, maka program akan menghasilkan kesalahan dengan melempar `IllegalArgumentException`, karena waktu tidak boleh bernilai negatif.

Selanjutnya nilai detik dibatasi dalam rentang satu hari dengan menggunakan operasi modulus terhadap jumlah detik dalam 24 jam. Setelah itu nilai detik dikonversi menjadi jam, menit, dan detik menggunakan operasi pembagian dan sisa pembagian.

2.3.2 Relasional

Relasional digunakan untuk membandingkan nilai jam satu dan nilai lainnya menggunakan *method* bawaan yaitu `compare()`. Misalkan terdapat dua variabel bilangan bulat x dan y , `compare(int x, int y)` membandingkan nilai x dan y . Jika nilai $x > y$, maka nilai kembaliannya akan bernilai positif ($0 < x$). Jika nilai $x = y$, maka nilai kembaliannya adalah 0. Jika $x < y$, maka nilai kembaliannya adalah akan bernilai negatif.

```
//---Relasional---
public int compareTo(Jam_2511532016 other){
    return Integer.compare(this.toSeconds(),other.toSeconds());}
```

Kode Program 2.8 – Potongan Program Perbandingan Nilai Jam

Tidak hanya itu, program ADT jam juga menyediakan fungsi khusus yang membandingkan apakah suatu jam memiliki nilai satuan waktu yang sama. Fungsi yang satu ini akan disebut “equals” dengan parameter bertipe suatu objek, dengan nama variabel “o”. Ia juga memastikan terlebih dahulu, apakah variabel “o” suatu objek dengan kelas Jam sebagai instansinya. Jika bukan, maka nilai kembaliannya otomatis false. Jika iya, maka ia akan membandingkan kesamaan dari setiap atribut utama dari kedua jam.

```
public boolean equals(Object o){
    if(!(o instanceof Jam_2511532016 j)) return false;
    return hh == j.hh && mm == j.mm && ss == j.ss;
}
```

Kode Program 2.9 – Potongan Program untuk Membandingkan Kesamaan Kedua Jam

2.3.3 Fungsi Aritmatika

Fungsi-fungsi aritmatika pada ADT jam berisikan penjumlahan waktu dalam satuan detik antara kedua jam, selisih jam dalam detik, detik satuan sebelum dan sesudah dari suatu jam, dan beberapa detik sesudah dan sebelum dari suatu jam.

```
//---Aritmatika---
public Jam_2511532016 plus(Jam_2511532016 other){return fromSeconds(this.toSeconds() + other.toSeconds());}
public Jam_2511532016 minus(Jam_2511532016 other){return fromSeconds(this.toSeconds() - other.toSeconds());}
public Jam_2511532016 nextSecond(){return fromSeconds(this.toSeconds() + 1);}
public Jam_2511532016 nextNSecond(int n){ return fromSeconds(this.toSeconds() + Math.max(0,n));}
public Jam_2511532016 prevSecond(){return fromSeconds(Math.floorMod(this.toSeconds() -1, 24*3600));}
public Jam_2511532016 prevNSecond(int n){ return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0,n), 24*3600));}
```

Kode Program 2.10 – Potongan-Potongan Program Untuk Operasi Aritmatika pada ADT Jam

Baris program pertama mengonversi satuan waktu yang ada pada jam satu dan jam lainnya menjadi detik, kemudian hasilnya dijumlahkan. Lalu hasil akhirnya akan membuat objek jam baru.

Baris program kedua melakukan hal yang mirip dengan baris program pertama, yang membedakan adalah baris program kedua mencari selisih dalam satuan detik antara jam, kemudian dikonversi menjadi objek jam yang baru.

Baris program ketiga membuat objek Jam baru yang menunjukkan waktu satu detik setelah waktu saat ini, dan baris program keempat ini menghasilkan objek Jam baru yang menunjukkan waktu n detik setelah waktu saat ini.

Baris program kelima membuat objek Jam baru yang menunjukkan waktu satu detik sebelum waktu saat ini, dan baris program keenam ini menghasilkan objek Jam baru yang menunjukkan waktu n detik sebelum waktu saat ini.

2.3.4 Fungsi Durasi

Fungsi ini mengembalikan durasi antara jam satu dan jam lainnya. Perbedaannya dengan fungsi operasi minus sebelumnya adalah, fungsi ini mengembalikan nilai dalam satuan detik, tidak dalam bentuk (jam, menit, detik).

```
//--- Durasi (detik) -- bisa negatif jika this > other? sesuai spesifikasi durasi (JAw, JAKh)
public static int durasiDetik(Jam_2511532016 JAw, Jam_2511532016 JAKh){
return JAKh.toSeconds() - JAw.toSeconds();}
```

Kode Program 2.11 – Potongan Program untuk Mencari Durasi Antar Dua Jam

2.3.5 Fungsi ToString

Fungsi ini meng-override fungsi bawaan dari Java. Fungsi dari Java pada aslinya mengubah suatu angka menjadi String. Tetapi, fungsi ini diubah oleh ADT Jam menjadi format jam.

```
//Overriding ke string
public String toString(){
return String.format("%02d:%02d:%02d", hh,mm,ss);}}
```

Kode Program 2.12 - Potongan Program untuk Mengubah Objek Jam Menjadi Format Jam Dalam Bentuk String

2.4 Kelas Driver Jam

Kelas driver adalah kelas yang umum dibuat untuk mengetes ADT yang telah dibuat. Pada kelas `JamDriver_2511532016`, telah dibuat barisan-barisan program. Pada kelas driver, dilakukan pembuatan beberapa objek `Jam` menggunakan konstruktor, kemudian objek tersebut digunakan untuk menguji method-method yang tersedia, seperti selektor, mutator, serta operasi aritmatika waktu.

Contoh pengujian yang dilakukan antara lain:

- Membuat objek jam
- Menampilkan objek jam yang telah dibuat
- Menjumlahkan dua jam
- Menambahkan jam sebanyak 20 detik
- Mencari jarak antara dua jam dalam satuan detik; dan
- Membandingkan dua jam

```

1 package pekan1_2511532016;
2 public class JamDriver_2511532016 {
3     public static void main(String[] args) {
4         Jam_2511532016 a = new Jam_2511532016(23, 59, 50);
5         Jam_2511532016 b = new Jam_2511532016(00, 00, 15);
6         System.out.println("a          = " + a);
7         System.out.println("b          = " + b);
8         System.out.println("a+b        = " + a.plus(b));
9         System.out.println("next 20s    = " + a.nextNSecond(20));
10        System.out.println("durasi(a,b) = " + Jam_2511532016.durasiDetik(a, b) );
11        System.out.println("a.compareTo(b) = " + a.compareTo(b));
12    }

```

Kode Program 2.12 – Kelas `JamDriver_251132016` Sebagai Pengujian

Keluaran dari program di atas adalah sebagai berikut.

```

<terminated> JamDriver_2511532016 [Java Application] C
a          = 23:59:50
b          = 00:00:15
a+b        = 00:00:05
next 20s    = 00:00:10
durasi(a,b) = -86375
a.compareTo(b) = 1

```

Gambar 2.1 – Keluaran dari Kelas `JamDriver_251132016`

2.5 Kelas Driver Lanjutan

Kelas driver ini lebih fleksibel dan dinamis karena dapat menerima input/masukan dari pengguna. Nama dari kelas ini adalah JamDriver2_2511532016.

2.5.1 Fungsi Pembuat Jam dari Input

```
private static Jam_2511532016 buatJamDariInput(Scanner sc) {
    int h, m, s;
    while(true) {
        System.out.print("Masukkan Jam (0-23)   :");
        h = sc.nextInt();
        System.out.print("Masukkan Menit (0-59) :");
        m = sc.nextInt();
        System.out.print("Masukkan Detik (0-59) :");
        s = sc.nextInt();
        //Memanggil method static isValid dari kelas Jam
        if (Jam_2511532016.isValid(h, m, s)) {
            return new Jam_2511532016(h,m,s);
        }
        else System.out.println("[Error] Input tidak valid! Silahkan ulangi.\n");
    }
}
```

Kode Program 2.13 – Fungsi untuk Membuat Jam

Fungsi di atas akan menerima input sebagai parameter dengan nama variabel sc, kemudian menginisialisasi tiga variabel; h, m, dan s.

Input pertama meminta jam dari objek jam, kemudian menitnya, dan terakhir detiknya.

Kemudian perlu dicek kesahan jam yang baru dibuat tersebut melalui fungsi statis dari kelas Jam_2511532016.

2.5.2 Blok Program Inisialisasi Dua Objek Jam

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    System.out.println("=== Program Driver Objek Jam ===");
    // 1. Input Jam Pertama
    System.out.println("\n--- Input Jam 1 ---");
    Jam_2511532016 j1 = buatJamDariInput(input);

    // 2. Input Jam Kedua
    System.out.println("\n--- Input Jam 2 ---");
    Jam_2511532016 j2 = buatJamDariInput(input);
}
```

Kode Program 2.14 – Potongan Program Inisialisasi Objek Jam Baru

Kode program di atas akan menginisialisasi objek Jam baru dengan nama j1 dan j2, dan menerima input melalui fungsi buatJamDariInput(). Keluaran dari potongan kode program tersebut adalah sebagai berikut jika diberikan input 12,45,10 untuk jam pertama dan 10, 25, 30 untuk jam ke dua.

```
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23)   :12
Masukkan Menit (0-59) :45
Masukkan Detik (0-59) :10

--- Input Jam 2 ---
Masukkan Jam (0-23)   :10
Masukkan Menit (0-59) :25
Masukkan Detik (0-59) :30
```

Gambar 2.2 – Keluaran dari Potongan Program Inisialisasi

2.5.3 Blok Program Menampilkan Data

```
//3. Menampilkan Data
System.out.println("\n--- Hasil Operasi ---");
System.out.println("Jam 1 (String)      : " + j1.toString());
System.out.println("Jam 2 (String)      : " + j2.toString());
System.out.println("Jam 1 (String)      : " + j1.toSeconds());
System.out.println("Jam 2 (String)      : " + j2.toSeconds());
```

Kode Program 2.15 – Potongan Program untuk Menampilkan Data dari Jam

Blok program di atas mengubah kedua objek jam menjadi dua bentuk, yaitu dalam bentuk format jam melalui baris ketiga dan keempat, dan dalam satuan detik melalui baris kelima dan keenam. Keluaran dari blok program tersebut jika digunakan input yang sama dengan sebelumnya adalah sebagai berikut.

```
--- Hasil Operasi ---
Jam 1 (String)      : 12:45:10
Jam 2 (String)      : 10:25:30
Jam 1 (String)      : 45910
Jam 2 (String)      : 37530
```

Gambar 2.3 – Keluaran dari Blok Program Menampilkan Data

2.5.4 Blok Program Operasi Rasional (Perbandingan)

```
// 4. Operasi Relasional (Perbandingan)
int perbandingan = j1.compareTo(j2);
if(perbandingan > 0) {
    System.out.println("Status      : Jam 1 lebih lambat (setelah) Jam 2");
}
else if(perbandingan < 0) {
    System.out.println("Status      : Jam 2 lebih lambat (setelah) Jam 1");
}
else System.out.println("Status      : Jam 1 dan Jam 2 sama persis");
```

Kode Program 2.16 - Blok Program untuk Operasi Perbandingan Jam

Untuk input yang sama dengan input sebelumnya, hasilnya adalah “Jam 1 lebih lambat (setelah) Jam 2”. Ini dapat dijelaskan karena jam pertama menunjukkan pukul 12, dan ini jelas lebih lambat dari pada jam kedua yang menunjukkan pukul 10.

2.5.5 Blok Operasi Aritmatika

```
// 5. Operasi Aritmatika
System.out.println("Durasi (J1 ke J2)      : " + Jam_2511532016.durasiDetik(j1, j2) + " detik");
Jam_2511532016 jNext = j1.nextSecond();
System.out.println("Jam 1 Detik Berikutnya : " + jNext);
Jam_2511532016 jPrev = j1.prevSecond();
System.out.println("Jam 1 Detik Berikutnya : " + jPrev);

//6. Operasi Penjumlahan Jam
Jam_2511532016 jHasilPlus = j1.plus(j2);
System.out.println("Hasil J1 + J2      : " + jHasilPlus);
input.close();
System.out.println("\nProgram Selesai.");
}
```

Kode Program 2.16 - Blok Program Operasi Aritmatika

Kode program di atas digunakan untuk melakukan operasi aritmatika sederhana seperti penjumlahan dan pengurangan antar objek jam.

Baris program kedua mencari berapa perbedaan durasi antara jam pertama dan kedua menggunakan fungsi statis `durasiDetik` dari kelas `Jam_2511532016`.

Dua baris selanjutnya mengoperasikan dan menampilkan detik selanjutnya dari jam pertama, dan dua baris setelahnya melakukan hal yang sama untuk jam kedua.

Terakhir, dibuat suatu objek jam baru dengan menjumlahkan kedua objek jam. Program pun selesai.

Output dari blok operasi aritmatika objek jam tersebut adalah sebagai berikut.

```
Durasi (J1 ke J2)      : -8380 detik
Jam 1 Detik Berikutnya : 12:45:11
Jam 1 Detik Berikutnya : 12:45:09
Hasil J1 + J2          : 23:10:40

Program Selesai.
```

Gambar 2.4 – Hasil dari Blok Program Aritmatika Objek Jam

2.6 Latihan Membuat ADT Mobil

2.6.1 Atribut

```
public class Mobil_2511532016 {
    //Atribut
    private String nama;
    private int tahun;
    private int cc;
    private int harga;
    private String merk;
```

Atribut dasar dari sebuah mobil adalah nama dari mobil, tahun keluaran, CC, harga dan merek mobil tersebut.

2.6.2 Validator dan Konstruktor

```
//Validator
private static boolean isValid(String n, int t, int cc, int h, String m) {
    return !(n.isBlank()) && !(t < 0) && !(cc < 0) && !(h < 0) && !(m.isBlank());
}

//Konstruktor
public Mobil_2511532016(String nama, int tahun, int cc, int harga, String merk) {
    if(isValid(nama, tahun, cc, harga, merk)) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }
}
```

Validator untuk memastikan atribut yang masuk sudah benar, dan konstruktor untuk membuat objek mobil.

2.6.3 Selektor dan Mutator

```
//Selektor
public String getNama(){return nama;}
public int getTahun(){return tahun;}
public int getCC(){return cc;}
public int getHarga(){return harga;}
public String getMerk(){return merk;}

//Mutator
public void setNama(String n) {this.nama = n;}
public void setTahun(int t) {this.tahun = t;}
public void setCC(int cc) {this.cc = cc;}
public void setHarga(int cc) {this.cc = cc;}
public void setMerk(String m) {this.merk = m;}
```

Selektor digunakan ketika kita ingin mengambil nilai dari salah satu atribut mobil, dan mutator digunakan untuk mengganti nilai dari suatu atribut mobil.

2.6.4 Method-Method Tambahan

```
43 //Method-Method Tambahan
44 private static Mobil_2511532016[] mobilMobilan = new Mobil_2511532016[100];
45 private static int n = 0;
46
47 //Menambahkan Mobil
48 public static void tambahMobil(Mobil_2511532016 M) {
49     if(n < mobilMobilan.length)
50         mobilMobilan[n] = M;
51     n++;
52 }
53 //Menghapus Mobil Berdasarkan Index
54 public static void hapusMobil(int idx){
55     if(idx >= 0 && idx < n){
56         for(int i = idx; i < n-1; i++){
57             mobilMobilan[i] = mobilMobilan[i+1];
58         }
59         mobilMobilan[n-1] = null;
60         n--;
61     }
62 }
63 //Menampilkan Mobil-Mobil
64 public static void tampilkanMobilMobilan(){
65     for(int i = 0; i < n; i++){
66         Mobil_2511532016 m = mobilMobilan[i];
67         System.out.println(
68             m.getNama()+" "+
69             m.getMerk()+" "+
70             m.getTahun()+" "+
71             m.getCC()+"cc "+
72             m.getHarga()
73         );
74     }
75 }
76
77 }
```

Method-method tambahan terdiri dari menambahkan, menghapus, dan menampilkan mobil-mobil.

Untuk menambah mobil, harus dipastikan terlebih dahulu apakah mobil tersebut dapat masuk ke dalam kontainer.

Untuk menghapus mobil, harus dipastikan apakah indeks bernilai non negatif dan lebih kecil dari ukuran kontainer saat ini. Setelah dihapus, ukuran kontainer diperkecil.

Mobil-mobil dapat ditampilkan dengan urutan nama, merek, tahun, CC, dan harga mereka

2.7 Kelas Driver MobilDriver_2511532016

```

3 public class MobilDriver_2511532016 {
4
5     public static void main(String[] args) {
6
7         Mobil_2511532016 M1 = new Mobil_2511532016("Xpander", 2021, 1500, 270000000, "Mitsubishi");
8         Mobil_2511532016 M2 = new Mobil_2511532016("Civic", 2020, 1800, 500000000, "Honda");
9         Mobil_2511532016 M3 = new Mobil_2511532016("Avanza", 2018, 1500, 200000000, "Toyota");
10        Mobil_2511532016 M4 = new Mobil_2511532016("MobilOwi", 2019, 0, 19000000, "AsliSolo");
11
12
13        Mobil_2511532016.tambahMobil(M1);
14        Mobil_2511532016.tambahMobil(M2);
15        Mobil_2511532016.tambahMobil(M3);
16        Mobil_2511532016.tambahMobil(M4);
17
18        System.out.println("=== Daftar Mobil ===");
19        Mobil_2511532016.tampilkanMobilMobilan();
20
21        System.out.println("\n=== Hapus Mobil Index 1 ===");
22        Mobil_2511532016.hapusMobil(3);
23
24        System.out.println("\n=== Daftar Mobil Setelah Dihapus ===");
25        Mobil_2511532016.tampilkanMobilMobilan();
26    }
27 }

```

Pertama, dibuat empat objek mobil baru dengan atribut mereka masing-masing. Kemudian, mobil-mobil tersebut dimasukkan ke dalam kontainer statis pada kelas ADT Mobil_2511532016. Lalu, mobil-mobil yang dimasukkan tadi ditampilkan ke layar melalui fungsi Mobil_2511532016.tampilkanMobilMobilan(). Lalu, mobil dengan indeks ke-3 dihapus, pada kasus ini yaitu mobil dengan nama "MobilOwi". Lalu, diperlihatkan kembali daftar mobil setelah penghapusan satu mobil.

BAB III

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Abstract Data Type (ADT) merupakan konsep penting dalam pemrograman yang mendefinisikan suatu tipe data berdasarkan data yang direpresentasikan serta operasi yang dapat dilakukan terhadap data tersebut. ADT menekankan pada perilaku (spesifikasi) tanpa bergantung pada cara implementasi di dalam program.

Dalam perancangan ADT, terdapat beberapa komponen utama seperti konstruktor, validator, selektor, dan mutator yang berperan dalam pembuatan, validasi, pengambilan, serta perubahan data pada objek. Dengan adanya komponen tersebut, data dapat dikelola secara lebih terstruktur dan aman.

Selain itu, ADT juga membedakan antara spesifikasi dan implementasi, di mana spesifikasi menjelaskan apa yang dapat dilakukan oleh suatu tipe data, sedangkan implementasi menjelaskan bagaimana operasi tersebut direalisasikan. Pemisahan ini memungkinkan program menjadi lebih modular dan mudah dikembangkan.

DAFTAR PUSTAKA

- [1]Oracle. *Java Documentation*. <https://docs.oracle.com/javase/>